

Módulo 3: Algoritmos de Machine Learning - Evaluación Práctica

Este documento contiene 10 ejercicios prácticos diseñados para evaluar y fortalecer las capacidades cognitivas en la implementación de algoritmos de clasificación utilizando Python, siguiendo la Taxonomía de Bloom.

Fundamentos Teóricos: SVM, Naive Bayes y Análisis Discriminante

En el desarrollo de modelos de clasificación, la elección del algoritmo no es arbitraria; responde a la naturaleza de los datos y los objetivos del negocio. A continuación, se detallan las diferencias clave y los criterios técnicos de selección.

1. Support Vector Machines (SVM)

Las SVM buscan encontrar el **hiperplano óptimo** que maximiza el margen entre las clases. Es un algoritmo de base geométrica.

- **Puntos fuertes:** Excelente rendimiento en espacios de alta dimensionalidad (más características que muestras) y capacidad de manejar datos no lineales mediante el **Kernel Trick**.
- **Cuándo elegirlo:** Cuando se busca la máxima precisión en datos complejos y limpios, y se dispone de tiempo de cómputo suficiente para el entrenamiento.

2. Naive Bayes

Basado en el **Teorema de Bayes**, asume que las características son independientes entre sí (supuesto "ingenuo"). Es un modelo probabilístico generativo.

- **Puntos fuertes:** Extremadamente rápido, eficiente en memoria y sorprendentemente robusto ante la violación de sus supuestos. Proporciona una probabilidad de pertenencia a la clase.
- **Cuándo elegirlo:** Procesamiento de texto (NLP), sistemas en tiempo real o cuando se necesita un modelo base (baseline) rápido con pocos datos.

3. Análisis Discriminante (LDA)

Busca proyectar los datos en un espacio de menor dimensión maximizando la separabilidad entre clases. Asume que los datos siguen una **distribución normal** y comparten una matriz de covarianza.

- **Puntos fuertes:** Además de clasificar, funciona como técnica de reducción de dimensionalidad. Muy estable si los supuestos estadísticos se cumplen.
 - **Cuándo elegirlo:** Cuando las clases están bien separadas, el dataset es pequeño/mediano y se requiere una técnica computacionalmente ligera y fácil de interpretar.
-

Criterios de Selección según Factores Clave

Factor	Sugerencia de Algoritmo	Razón Técnica
Datos No Lineales	SVM (RBF/Poly)	Los kernels transforman los datos para hacerlos linealmente separables.
Velocidad de Respuesta	Naive Bayes	Su cálculo es una simple multiplicación de probabilidades.
Alta Dimensionalidad	SVM	Está diseñado para evitar el "sobreajuste" en espacios con muchas variables.
Visualización/Reducción	LDA	Proyecta los datos maximizando la distancia entre las medias de las clases.
Robustez ante Outliers	Naive Bayes	Al ser probabilístico, las muestras extremas tienen menos impacto en la media global.

Métricas de Evaluación en Clasificación

La evaluación de un modelo de clasificación es crítica. No basta con saber "cuánto acertó"; debemos entender "cómo falló". Dependiendo del problema, una métrica puede ser mucho más relevante que otra.

1. Métricas Globales y la Matriz de Confusión

La mayoría de los algoritmos de este módulo se evalúan mediante la comparación de las predicciones frente a los valores reales en una matriz de confusión.

- **Exactitud (Accuracy):** Es el porcentaje total de aciertos sobre el total de casos. Es útil solo si las clases están balanceadas (misma cantidad de ejemplos de cada tipo).
 - *Interpretación:* "El modelo acertó en 9 de cada 10 casos totales".
- **Precisión (Precision):** De todas las veces que el modelo predijo una clase (ej. "Es Spam"), ¿cuántas veces acertó realmente? Se enfoca en minimizar los **Falsos Positivos**.
 - *Ejemplo:* En un filtro de spam, una precisión alta asegura que no perderemos correos importantes marcados erróneamente como basura.
- **Sensibilidad o Exhaustividad (Recall):** De todos los casos reales que existían de una clase (ej. "Enfermos"), ¿cuántos logró detectar el modelo? Se enfoca en minimizar los **Falsos Negativos**.
 - *Ejemplo:* En un diagnóstico de cáncer, es vital tener un Recall cercano al 100% para no dejar a ningún paciente enfermo sin tratamiento.
- **F1-Score:** Es la media armónica entre la Precisión y el Recall. Se utiliza cuando necesitamos un equilibrio entre ambas métricas y tenemos clases desbalanceadas.

2. Métricas Específicas por Algoritmo

Cada algoritmo posee métricas o conceptos internos que ayudan a interpretar su funcionamiento:

Algoritmo	Métrica o Concepto Clave	Cómo interpretarlo
-----------	--------------------------	--------------------

Algoritmo	Métrica o Concepto Clave	Cómo interpretarlo
SVM	Márgenes y Vectores de Soporte	Un margen más ancho entre clases indica una mayor robustez y capacidad de generalización del modelo.
Naive Bayes	Probabilidad Posterior	Permite conocer el grado de certeza (ej. "Hay un 85% de probabilidad de que este mensaje sea Spam").
LDA	Varianza Explicada	Indica cuánta información de separabilidad de clases se mantiene tras reducir las dimensiones.
KNN	Distancia (Métrica)	Define la similitud. Una distancia pequeña indica que el dato es muy parecido a sus vecinos conocidos.
Árboles / R. Forest	Impureza de Gini / Importancia	Gini mide qué tan "mezcladas" están las clases en un nodo. La importancia indica qué variable decide más.
Redes Neuronales	Log-Loss (Cross-Entropy)	Mide la penalización por predicciones erróneas. A menor pérdida, mejor aprendizaje de la red.

3. Ejemplo de Interpretación: El Caso del Fraude Bancario

Imagina un modelo que detecta fraude en tarjetas de crédito:

- Si el modelo tiene **Alta Precisión**, significa que cuando bloquea una tarjeta por fraude, casi siempre hay un fraude real (poca molestia al cliente legítimo).
- Si el modelo tiene **Alto Recall**, significa que el banco está capturando casi todos los fraudes que ocurren (poca pérdida económica para el banco).
- **El Dilema:** Subir el Recall suele bajar la Precisión (el banco se vuelve más "desconfiado" y bloquea más tarjetas legítimas por error). El especialista debe elegir el punto de equilibrio según la estrategia del negocio.

Ejercicio 1 - Clasificación Básica con SVM (Iris)

Objetivo: Aplicar los conceptos fundamentales de Support Vector Machines para una clasificación multiclase.

Enunciado del Reto: Un equipo de botánicos necesita automatizar la identificación de la especie *Iris* basándose en medidas físicas. Tu tarea es cargar el dataset Iris de Scikit-Learn, dividirlo en entrenamiento y prueba, y entrenar un modelo SVM con kernel lineal para predecir la especie de una muestra desconocida.

Solución en Python:

```
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
import numpy as np

# 1. Cargar el dataset
```

```
iris = datasets.load_iris()
X, y = iris.data, iris.target

# 2. Dividir en entrenamiento (70%) y prueba (30%)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

# 3. Crear y entrenar el modelo (Kernel Lineal)
modelo_svm = SVC(kernel='linear')
modelo_svm.fit(X_train, y_train)

# 4. Evaluación básica
precision = modelo_svm.score(X_test, y_test)
print(f"Precisión del modelo: {precision:.2f}")

# 5. Predicción de una nueva muestra desconocida
nueva_flor = np.array([[5.1, 3.5, 1.4, 0.2]])
prediccion = modelo_svm.predict(nueva_flor)
print(f"La flor pertenece a la especie: {iris.target_names[prediccion][0]}")
```

Justificación: El alumno demuestra capacidad de aplicación al integrar el flujo básico de Scikit-Learn (Carga, Split, Fit, Predict) en un problema de clasificación estándar.

Ejercicio 2 - Probabilidades con Naive Bayes (Iris)

Objetivo: Aplicar modelos probabilísticos para entender la pertenencia a clases.

Enunciado del Reto: En un estudio genético, se requiere no solo clasificar la especie, sino conocer el nivel de confianza de la predicción. Implementa el algoritmo Gaussian Naive Bayes sobre el dataset Iris y muestra las probabilidades exactas de que una flor con medidas [6.7, 3.1, 4.4, 1.4] pertenezca a cada una de las tres categorías.

Solución en Python:

```
from sklearn.naive_bayes import GaussianNB
from sklearn import datasets

# Carga de datos
iris = datasets.load_iris()
X, y = iris.data, iris.target

# Inicializar y entrenar el clasificador Naive Bayes
gnb = GaussianNB()
gnb.fit(X, y)

# Definir la muestra a evaluar
muestra = [[6.7, 3.1, 4.4, 1.4]]

# Obtener las probabilidades de pertenencia a cada clase
probabilidades = gnb.predict_proba(muestra)
```

```
print("Probabilidades por especie:")
for i, nombre in enumerate(iris.target_names):
    print(f"- {nombre.capitalize()}: {probabilidades[0][i]:.4f}")
```

Justificación: La solución requiere que el alumno utilice `predict_proba`, demostrando que comprende que Naive Bayes se basa en el teorema de Bayes para asignar pesos probabilísticos.

Ejercicio 3 - Diagnóstico Médico con SVM (Breast Cancer)

Objetivo: Aplicar técnicas de clasificación en un entorno de alta criticidad (Salud).

Enunciado del Reto: Un hospital digital desea una herramienta de soporte para diagnosticar cáncer de mama (Maligno/Benigno). Utiliza el dataset UCI Breast Cancer para entrenar un modelo SVM. Asegúrate de evaluar el modelo con el conjunto de prueba y mostrar el porcentaje de aciertos.

Solución en Python:

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

# 1. Carga del dataset de cáncer de mama UCI
cancer = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(cancer.data, cancer.target,
test_size=0.2, random_state=42)

# 2. Configuración del modelo SVC
# Usamos parámetros por defecto para observar el rendimiento base
clf = SVC()
clf.fit(X_train, y_train)

# 3. Predicción y evaluación
y_pred = clf.predict(X_test)
acc = accuracy_score(y_test, y_pred)
print(f"Precisión en el diagnóstico médico: {acc*100:.2f}%")
```

Justificación: Evalúa la transferencia de conocimientos de un dataset simple (Iris) a uno con más dimensiones (30 características) y un impacto social real.

Ejercicio 4 - Reducción de Dimensiones con LDA

Objetivo: Analizar cómo el Análisis Discriminante Lineal (LDA) ayuda a separar clases visualmente.

Enunciado del Reto: Visualizar datos en 4 dimensiones (Iris) es complejo. Utiliza LDA para reducir el dataset a 2 componentes principales que maximicen la separabilidad de las clases y genera un gráfico de dispersión (scatter plot) para observar cómo se agrupan las especies.

Solución en Python:

```
import matplotlib.pyplot as plt
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn import datasets

# Carga de datos
iris = datasets.load_iris()
X, y = iris.data, iris.target

# Reducción a 2 dimensiones usando LDA
# LDA busca maximizar la distancia entre medias de clases y minimizar la varianza interna
lda = LinearDiscriminantAnalysis(n_components=2)
X_lda = lda.fit(X, y).transform(X)

# Visualización de los resultados
plt.figure(figsize=(8, 6))
colors = ['navy', 'turquoise', 'darkorange']
for i, color, name in zip([0, 1, 2], colors, iris.target_names):
    plt.scatter(X_lda[y == i, 0], X_lda[y == i, 1], color=color, alpha=.8,
label=name)

plt.legend(loc='best', shadow=False, scatterpoints=1)
plt.title('LDA: Proyección del dataset Iris en 2D')
plt.show()
```

Justificación: El alumno analiza la capacidad de LDA para proyectar datos preservando la información de clase, diferenciándolo de un simple análisis estadístico.

Ejercicio 5 - Sensibilidad al Escalamiento en SVM

Objetivo: Analizar la importancia del preprocesamiento de datos en algoritmos basados en distancias.

Enunciado del Reto: Los modelos SVM son extremadamente sensibles a la escala de las variables. Demuestra este impacto comparando el rendimiento de un modelo SVM entrenado con los datos de Cáncer de Mama "crudos" frente a uno entrenado con los datos normalizados usando `StandardScaler`.

Solución en Python:

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler

# Datos
data = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(data.data, data.target,
random_state=1)
```

```
# 1. Modelo sin escalado
svm_raw = SVC().fit(X_train, y_train)
score_raw = svm_raw.score(X_test, y_test)

# 2. Modelo con escalado
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

svm_scaled = SVC().fit(X_train_scaled, y_train)
score_scaled = svm_scaled.score(X_test_scaled, y_test)

print(f"Rendimiento sin escalado: {score_raw:.4f}")
print(f"Rendimiento con escalado: {score_scaled:.4f}")
```

Justificación: Analiza la importancia del preprocesamiento, comprendiendo que la arquitectura del algoritmo (márgenes) depende de la magnitud de los vectores.

Ejercicio 6 - Selección de Kernel en SVM

Objetivo: Evaluar la eficacia de diferentes fronteras de decisión (lineales vs no lineales).

Enunciado del Reto: En el dataset de cáncer de mama, las relaciones entre variables pueden no ser lineales. Evalúa el rendimiento de un SVM con kernel 'linear' frente a uno 'rbf' utilizando validación cruzada (Cross-Validation) para determinar cuál generaliza mejor.

Solución en Python:

```
from sklearn.datasets import load_breast_cancer
from sklearn.svm import SVC
from sklearn.model_selection import cross_val_score

cancer = load_breast_cancer()

# Definimos los modelos a comparar
modelos = {
    "SVM Lineal": SVC(kernel='linear'),
    "SVM RBF (No lineal)": SVC(kernel='rbf')
}

print("Resultados de Validación Cruzada (CV=5):")
for nombre, modelo in modelos.items():
    puntuaciones = cross_val_score(modelo, cancer.data, cancer.target, cv=5)
    print(f"- {nombre}: {puntuaciones.mean():.4f} (+/- {puntuaciones.std() * 2:.4f})")
```

Justificación: Al evaluar resultados estadísticos, el alumno decide críticamente qué configuración matemática es superior para un conjunto de datos específico.

Ejercicio 7 - Análisis de Errores con Matriz de Confusión

Objetivo: Evaluar el coste de los errores en un modelo de clasificación.

Enunciado del Reto: No todos los errores pesan igual. En el diagnóstico de cáncer, un Falso Negativo es mucho más grave que un Falso Positivo. Genera una Matriz de Confusión para el clasificador de cáncer de mama e identifica cuántos casos malignos fueron erróneamente clasificados como benignos.

Solución en Python:

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
import matplotlib.pyplot as plt

data = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(data.data, data.target,
test_size=0.3, random_state=0)

# Entrenar modelo (usamos kernel lineal por su estabilidad en este dataset)
clf = SVC(kernel='linear').fit(X_train, y_train)
y_pred = clf.predict(X_test)

# Generar y mostrar la matriz
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
display_labels=data.target_names)
disp.plot(cmap='Reds')
plt.title("Matriz de Confusión: Diagnóstico Oncológico")
plt.show()
```

Justificación: El alumno evalúa la utilidad real del modelo mediante el análisis detallado de la matriz, reconociendo la diferencia entre precisión y seguridad.

Ejercicio 8 - Clasificación Discriminante vs Probabilística

Objetivo: Evaluar las diferencias entre LDA y Naive Bayes en condiciones reales.

Enunciado del Reto: Entrena un clasificador LDA y uno Naive Bayes sobre el dataset Iris. Determina cuál de los dos comete menos errores en el conjunto de prueba y reflexiona sobre si la suposición de independencia de variables de Naive Bayes afecta los resultados.

Solución en Python:

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.naive_bayes import GaussianNB
from sklearn import datasets
```



```

from sklearn.metrics import classification_report

iris = datasets.load_iris()
X_train, X_test, y_train, y_test = train_test_split(iris.data, iris.target,
test_size=0.4, random_state=42)

# Entrenamiento
lda = LinearDiscriminantAnalysis().fit(X_train, y_train)
gnb = GaussianNB().fit(X_train, y_train)

# Reporte de resultados
print("--- Rendimiento LDA ---")
print(classification_report(y_test, lda.predict(X_test),
target_names=iris.target_names))

print("\n--- Rendimiento Naive Bayes ---")
print(classification_report(y_test, gnb.predict(X_test),
target_names=iris.target_names))

```

Justificación: El alumno evalúa supuestos teóricos comparando métricas de precisión, recall y F1-score para dos paradigmas distintos de clasificación.

Ejercicio 9 - Optimización de Hiperparámetros (C y Gamma)

Objetivo: Crear un proceso de sintonización (tuning) para encontrar la mejor configuración de un SVM.

Enunciado del Reto: El rendimiento de SVM depende de los parámetros **C** (regularización) y **gamma** (influencia de muestras). Crea un proceso automatizado usando **GridSearchCV** que pruebe múltiples combinaciones de estos parámetros sobre el dataset de cáncer de mama y reporte la mejor configuración encontrada.

Solución en Python:

```

from sklearn.model_selection import GridSearchCV
from sklearn.datasets import load_breast_cancer
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler

data = load_breast_cancer()
X_scaled = StandardScaler().fit_transform(data.data)

# Definir la rejilla de parámetros (Grid)
param_grid = {
    'C': [0.1, 1, 10, 100],
    'gamma': [1, 0.1, 0.01, 0.001],
    'kernel': ['rbf']
}

# Crear y ejecutar la búsqueda
grid = GridSearchCV(SVC(), param_grid, refit=True, verbose=0, cv=5)
grid.fit(X_scaled, data.target)

```

```
print(f"Mejores parámetros encontrados: {grid.best_params_}")
print(f"Mejor precisión obtenida: {grid.best_score_:.4f}")
```

Justificación: El alumno crea un sistema de búsqueda exhaustiva, demostrando que puede orquestar herramientas de optimización para mejorar modelos existentes.

Ejercicio 10 - Pipeline Integral de Machine Learning

Objetivo: Crear una solución de extremo a extremo (End-to-End) robusta y profesional.

Enunciado del Reto: Como consultor experto, debes crear un "Pipeline" que automatice todo el flujo de trabajo para nuevos datos de investigación floral: 1. Escale los datos, 2. Reduzca la dimensionalidad con LDA a 1 componente, y 3. Clasifique mediante SVM. Este pipeline debe ser capaz de entrenarse y predecir de forma atómica.

Solución en Python:

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.svm import SVC
from sklearn import datasets

# Cargar datos
iris = datasets.load_iris()
X, y = iris.data, iris.target

# Construcción del Pipeline Profesional
# El Pipeline asegura que el escalado y la reducción se apliquen consistentemente
pipeline_floral = Pipeline([
    ('escalador', StandardScaler()),
    ('reductor_lda', LinearDiscriminantAnalysis(n_components=2)),
    ('clasificador_svm', SVC(kernel='poly', degree=3, C=1.0))
])

# Entrenamiento completo del flujo
pipeline_floral.fit(X, y)

# Simulación de llegada de nuevos datos
nuevos_datos = [[5.0, 3.6, 1.4, 0.2], [6.5, 3.0, 5.2, 2.0]]
predicciones = pipeline_floral.predict(nuevos_datos)

print("Predicciones del Pipeline para nuevas muestras:")
for i, pred in enumerate(predicciones):
    print(f" Muestra {i+1}: {iris.target_names[pred].upper()}")
```

Justificación: Demuestra la capacidad de creación al ensamblar múltiples componentes técnicos en una solución arquitectónica coherente, escalable y reproducible.

